

Tendências de Motores de Jogos

Semana 13

Áudio de Eventos e Profiling — Produção Técnica

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 4 — Produzir como um Pequeno Estúdio

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender áudio como resposta a eventos de gameplay já existentes, não como camada decorativa de final de produção
- Compreender Optimization e Profiling como etapa obrigatória de produção, capaz de identificar gargalos antes de qualquer entrega
- Integrar sons a ações já existentes no projeto (interação, passos, ambiente), sem alterar nenhum sistema de gameplay
- Executar profiling do próprio Vertical Slice, identificar ao menos um gargalo real e tratá-lo, justificando a escolha técnica

Como a semana está organizada



Encontro 1

Áudio de eventos — som como resposta a ações já existentes no projeto



Encontro 2

Optimization e Profiling — medir antes de otimizar, com Feedback formal de encerramento

Tendências de Motores de Jogos

ENCONTRO 1

Áudio de Eventos

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Um som precisa de um sistema próprio para existir no jogo?

Pense em tudo que já dispara uma reação no projeto sempre que o jogador interage, anda ou entra em uma nova área.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Áudio como resposta a um evento, não um sistema à parte

- Um som, em uma engine, não é um arquivo tocado isoladamente — é a resposta a um evento que já existe no fluxo de gameplay
- O mesmo Event Dispatcher que já dispara a interação via `BPI_Interactable` pode disparar um som, sem sistema novo
- Som pontual (disparado por um evento) é diferente de som contínuo/ambiente (associado a um espaço ou estado)
- Esta integração é deliberadamente tardia — ocorre depois que o gameplay já está estável desde os Módulos 2 e 3

DICA

Áudio comunica estado e reforça feedback: uma porta que range ao abrir confirma a interação tanto quanto a animação.

Onde o som se conecta ao que já existe

- Interação: o Event Dispatcher já existente no `InteractionComponent` dispara o som de portas, alavancas e baús
- Locomoção: a movimentação do `BP_Player` dispara o som de passos
- Ambiente: cada nível (Exploração, Dungeon) recebe som contínuo associado ao próprio espaço
- Sons mal gerenciados (excesso de sons simultâneos, sem prioridade ou distância de corte) também custam performance — antecipa o Encontro 2

Áudio de eventos: Unreal × Unity

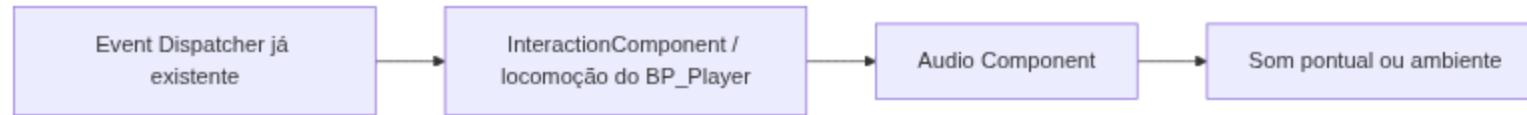
Unreal

Sound Cue/Sound Base e Audio Component, disparados a partir de Event Dispatchers já existentes (`InteractionComponent`)

Unity

`AudioSource` e `AudioClip`, disparados a partir de eventos de script (`OnTriggerEnter`, chamadas diretas de método)

Do evento existente ao som disparado



Demonstração: som de interação e som de passos

O professor associa um som ao evento já disparado pelo Event Dispatcher do `InteractionComponent` e um som de passos à locomoção do `BP_Player`, sem alterar a lógica de gameplay existente.

Resultado esperado: interação e locomoção já existentes passam a soar, sem nenhuma mudança na lógica que as dispara.

Imagem sugerida

Objetivo: mostrar a conexão entre um Event Dispatcher já existente e o disparo de um Audio Component.

*Enquadramento: captura de tela do grafo de Blueprint do **InteractionComponent** na Unreal Engine, com o nó de Event Dispatcher e o nó de Play Sound conectados na mesma sequência de execução.*

Elementos presentes: nó de Event Dispatcher já existente; nó de Audio Component/Play Sound anexado à mesma linha de execução; comentário indicando "nenhuma lógica nova".

Destaque visual: a linha de execução única que liga o evento de interação ao som.

Legenda sugerida: "O som reage ao mesmo evento que já dispara a interação."

Boas práticas

✓ BOAS PRÁTICAS

- Conectar o som ao Event Dispatcher já existente, nunca criar um gatilho novo só para o áudio
- Organizar os assets sonoros em `Audio/`, conforme `PROJECT_ARCHITECTURE.md`
- Definir prioridade e distância de corte para sons de ambiente, evitando excesso de sons simultâneos

Laboratório do Encontro 1

Cada grupo integra som às suas próprias ações de interação (portas, alavancas, baús), à locomoção do próprio `BP_Player` e à ambientação de seus níveis de Exploração e Dungeon, organizando os assets em `Audio/`.

OBJETIVOS

Critério de sucesso: som integrado a pelo menos uma ação de interação, à locomoção do `BP_Player` e a um elemento de ambiente do nível, organizados em `Audio/`, sem alterar nenhum sistema de gameplay existente.

Fechando o Encontro 1

- Áudio é resposta a um evento de gameplay já existente, não um sistema autônomo
- Interação, locomoção e ambiente de cada grupo já soam, sem alteração de nenhuma lógica de gameplay
- Próximo encontro: a mesma disciplina de medir antes de agir, agora aplicada à performance do projeto

Tendências de Motores de Jogos

ENCONTRO 2

Optimization e Profiling

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Como saber qual parte do projeto está custando performance?

Pense no que aconteceria se você tentasse otimizar sem antes medir onde o tempo de frame está sendo gasto.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Medir antes de otimizar

- Nenhuma decisão de otimização é válida sem medição prévia — otimizar por intuição tende a gastar esforço na parte errada
- A Unreal expõe essa medição por ferramentas de profiling: Stat commands e Unreal Insights/Session Frontend, conforme disponibilidade da versão 5.6
- Essas ferramentas revelam onde o tempo de frame é consumido: geometria, materiais, iluminação ou lógica de Blueprint
- A densidade de Foliage da Semana 12 é o primeiro candidato a gargalo mensurável, mas a discussão se amplia às demais camadas do semestre

DICA

Este encontro retoma diretamente a densidade de Foliage discutida na Semana 12 — agora com dados concretos de profiling, não apenas impressão visual.

Onde procurar o gargalo

- Fontes comuns de gargalo em um Vertical Slice deste porte: densidade de Foliage, draw calls, complexidade de Materials, iluminação dinâmica, lógica de Blueprint em Tick
- `stat fps`, `stat unit` e `stat scenerendering` expõem o custo de cada camada em tempo real
- Isolar variáveis uma de cada vez (ex.: ocultar temporariamente a Foliage) ajuda a confirmar a origem real do gargalo
- O gargalo real depende do que cada grupo construiu — não existe uma resposta única para toda a turma

Profiling: Unreal × Unity

Unreal

Stat commands e Unreal Insights/Session Frontend, expondo dados por sistema (renderização, materiais, Blueprint) em ferramentas complementares

Unity

Profiler integrado ao Editor, concentrando CPU, GPU, memória e renderização por frame em um único painel unificado

Tendências de Motores de Jogos

Da medição à otimização justificada



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Demonstração: profiling guiado com gargalo proposital

O professor usa stat commands (e Unreal Insights, se disponível) sobre uma cena de teste com um gargalo proposital de densidade de Foliage, mostrando como o dado de profiling aponta a causa antes de qualquer correção.

Resultado esperado: identificação clara de qual camada (geometria, materiais, iluminação ou Blueprint) está consumindo o tempo de frame, antes de qualquer mudança na cena.

Imagem sugerida

Objetivo: evidenciar a leitura de um dado de profiling antes e depois de uma otimização aplicada.

*Enquadramento: duas capturas de tela lado a lado do editor da Unreal Engine, mesma cena de teste, com **stat fps** / **stat unit** visível no canto superior.*

Elementos presentes: overlay de stat commands com valores de frame time; indicação visual da camada testada (ex.: Foliage oculta versus visível).

Destaque visual: a diferença numérica do frame time entre as duas capturas.

Legenda sugerida: "Nenhuma otimização é válida sem o dado que a justifica."

Boas práticas

✓ BOAS PRÁTICAS

- Medir com stat commands antes de propor qualquer otimização
- Isolar uma variável por vez ao investigar um gargalo, em vez de alterar várias camadas simultaneamente
- Registrar o dado de profiling que motivou cada otimização, para sustentar a justificativa técnica no Feedback

Laboratório/Desafio: profiling do próprio projeto

Cada grupo executa profiling do próprio Vertical Slice, identifica um gargalo real (geometria, materiais, iluminação ou lógica de Blueprint) e o trata, justificando tecnicamente a escolha com base nos dados coletados.

OBJETIVOS

Critério de sucesso: profiling executado, ao menos um gargalo real identificado com dados concretos, gargalo tratado e justificativa técnica coerente apresentada no Feedback formal, sem regressão em nenhum sistema já construído.

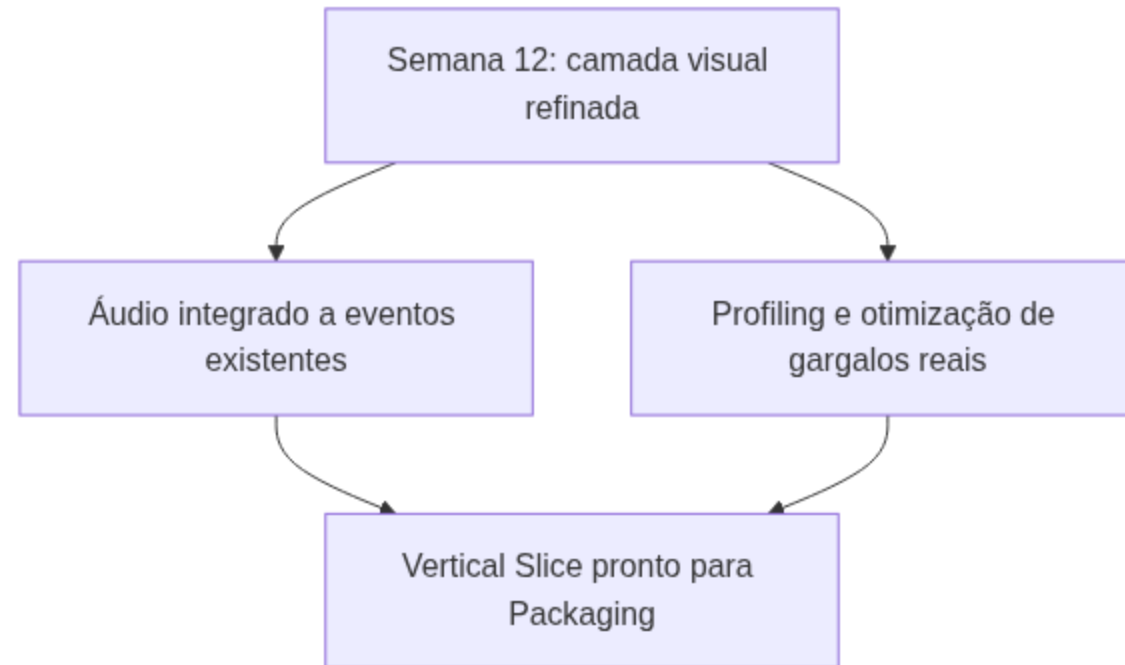
Feedback Formal de Encerramento — Áudio e Otimização

Avaliação formal das otimizações realizadas por cada grupo, com ênfase na qualidade da medição (profiling real, não suposição), na pertinência da escolha de otimização e na clareza da justificativa técnica apresentada.

ATENÇÃO

Otimizações propostas sem o dado de profiling correspondente, e justificativas baseadas em suposição em vez de medição, são os principais pontos de atenção deste Feedback.

Onde a Semana 13 se encaixa no Vertical Slice



Resumo da Semana 13

- Áudio é resposta a um evento de gameplay já existente, não um sistema autônomo adicionado ao final
- Otimização exige medição prévia — profiling é etapa obrigatória, não correção de emergência
- Áudio integrado a interação, locomoção e ambiente; ao menos um gargalo real identificado e tratado
- Feedback formal de encerramento concluído, com justificativa técnica registrada por grupo

Checklist Final do Encontro

OBJETIVOS

- Som integrado a interação, locomoção do `BP_Player` e ambiente, organizado em `Audio/`
- Profiling executado com stat commands (e Unreal Insights, se disponível)
- Ao menos um gargalo real identificado e tratado, com justificativa técnica registrada
- `HealthComponent`, `InteractionComponent`, `InventoryComponent`, `BPI_Interactable`, `WBP_HUD`, `BP_Enemy`, Material Instances e Foliage intactos
- Feedback formal de encerramento concluído

Próximos passos

DICA

A Semana 14 encerra a Unidade IV com o pipeline de Packaging do Vertical Slice final — configurações, plataformas-alvo e build de produção — diretamente dependente do trabalho de otimização desta semana: um projeto com gargalos não tratados compromete o empacotamento e o Playtest cruzado entre grupos.

Leitura recomendada: Unreal Engine — Audio Overview e Optimization Guide (Epic Games Documentation).